



Development of a new framework for high performance volunteer computing

Ehab Saleh¹ · B. S. Chandrasekar²

Received: 28 July 2024 / Accepted: 7 November 2024
© Bharati Vidyapeeth's Institute of Computer Applications and Management 2024

Abstract The majority of Volunteer Computing (VC) projects are based on the Berkeley Open Infrastructure for Network Computing (BOINC) framework. BOINC is an open-source middleware system designed to support a variety of volunteer computing projects across multiple scientific disciplines, including molecular biology, mathematics, cryptography, linguistics, and astrophysics. However, it is worth noting that BOINC primarily supports a client-server network topology. This setup has a potential drawback known as a Single Point of Failure (SPoF). In this context, if the server fails, the entire network may fail, resulting in the failure of ongoing research projects. In this paper, we present the design of a new Peer-to-Peer (P2P) volunteer computing framework that addresses the SPoF issue. This framework allows users to create their own computing environment simply by sharing a project invitation. Those who accept the invitation become members of the network and contribute their computing resources. The experimental findings from our simulations highlight the promising capabilities of this framework, particularly in terms of load balancing and computing speedup.

Keywords High performance computing · Distributed systems · Volunteer computing

1 Introduction

The massive computational processes in today's computer applications require the use of high-performance computing environments. There are problems in mathematics, physics, astronomy, and other scientific fields that require urgent results in a reasonable amount of time that cannot be achieved with our traditional personal computers.

Supercomputers and distributed systems are two main high-performance computing environments available to researchers for tackling complex problems. However, both environments involve significant costs and power consumption that must be addressed for effective operation.

Volunteer Computing (VC) refers to the concept of utilizing unused computing power on volunteers' devices without significantly affecting the device's performance or normal power consumption. This idea was motivated by researchers seeking alternative computing platforms to conduct their projects without incurring the high costs and enormous power consumption typically associated with traditional high-performance computing environments.

Before the term Volunteer Computing was coined, the concept of utilizing machines during their idle time appeared in the Network of Workstations (NoW) project [2]. NoW refers to the ability to run parallel programs on workstation clusters during idle times without the need to purchase supercomputing infrastructure. It was successfully used to solve large parallel computational problems, achieving over 10 GFLOPS in 1997 and ranking among the world's top 200 fastest supercomputers. This concept of voluntary participation emerged in the mid-90s and, since then, multiple VC platforms have been introduced. Some of these platforms were created to address specific problems, while others were designed as generic frameworks capable of handling many projects across various scientific disciplines. Today, there

✉ Ehab Saleh
ehab.saleh@lrz.de

¹ Leibniz Supercomputing Centre, Munich, Germany

² Jain University, Bengaluru, India

are around 50 scientific projects that make use of the VC concept, with several of them built on the BOINC framework. [3]. These VC projects involve more than 900,000 machines. These machines have 560,000 GPUs and 5 million CPU cores, with a throughput of more than 100 peta-FLOPS on average [4].

All currently running BOINC-based VC projects use a client-server network architecture. In this setup, a single server is responsible for project scheduling and managing connections with all the participating volunteers. However, if the server fails, the entire project can fail. Additionally, users who want to start their own VC project must acquire and pay for a server. If they do not choose a server with adequate technical specifications, it may struggle to handle numerous tasks and manage a large number of newly joined volunteers effectively.

We aim to introduce a high-performance VC framework that addresses the limitations of existing VC platforms. Instead of relying on a client-server architecture, our framework will employ a structured peer-to-peer (P2P) network where peers act as both servers and clients simultaneously. In other words, there is no centralized server responsible for distributing tasks and managing connections with all other clients. Instead, multiple peers can perform server tasks, but these responsibilities are limited to their direct peers, rather than the entire network of volunteers.

The rest of this paper is structured as follows: Sect. 2 provides an in-depth examination of the most used VC frameworks, with a particular focus on the architecture of the BOINC framework. Section 3 explores the architecture of the proposed framework. Section 4 describes the network topology and the characteristics of the computing resources that define the system. Section 5 presents the simulation results, analyzing efficiency and speedup in relation to task size. Sections 6 and 7 conclude the paper and suggest directions for future research.

2 Related works

The notion of voluntary contributing to address significant scientific challenges emerged initially within the Network of Workstations (NoW) project. This approach utilizing clusters of workstations connected by high-speed networks to execute parallel programs during idle time, thus eliminating the need to invest in supercomputing resources.

The first implementation of volunteer computing concepts on a real-world platform was in Great Internet Mersenne Prime Search (GIMPS). It was thought to be the largest research project and the first to use volunteer computing on the internet. It was founded in 1996 with two versions of code: the client-version and the server-version. GIMPS began to attract a large number of people to download the

client-version and join the project, and thousands of them were already actively participating by the end of the first year of its introduction. In November 1996, after only ten months since the launch of the project, GIMPS has discovered the 35th known Mersenne Prime that has the form $M_{1398269} = 2^{1398269} - 1$ [5].

New efforts have been ramped up to harness the CPU idle time of volunteer computers to solve high-throughput problems. One notable initiative is the Distributed.net project [6], which is managed by Distributed Computing Technologies, Inc. (DCTI).

We divide our survey of VC platforms into three main sections.: in Sect. 2.1 we dissect the most well-known open source frameworks, which were built on the Java programming language and were only aiming at Java-based distributed applications. Then in Sect. 2.2 we discuss the well-known VC platform BOINC. In Sect. 2.3, we go through other VC platforms that were introduced by individual groups.

2.1 Java-based volunteer computing

The Java programming language offers a wide array of libraries that enable developers to create parallel programs capable of running simultaneously across multiple devices. One of the pioneering platforms in this domain was ParaWeb [7], which was built using Java and leveraged the Java Virtual Machine (JVM) to execute code-interpreted across various CPU architectures. This represented a significant advancement in distributed applications, effectively addressing the challenge of diverse CPU architectures.

In general, there are two primary ways to achieve parallelism in Java. The first method involves extending the programming system without modifying the Java runtime environment. Alternatively, parallelism can be achieved using pre-built libraries, such as threads and synchronization. ParaWeb utilized both approaches through two mechanisms: the Java Parallel Runtime System (JPRS) [8] and the Java Parallel Class Library (JPCL) [9].

Atlas [10] is yet another distributed platform that uses Java to provide the community with a computing environment comprised of thousands of connected machines. Atlas's programming model is based on multi-threading, which was derived from the Cilk multi-threaded runtime system [11]. This programming model abstracts all computing resources, allowing the runtime system to manage them in the highly dynamic manner required for global-scale execution.

Figure 1 depicts the basic architecture of Atlas. The client first requests the address of a relevant server from the manager. Upon receiving the address, the client can then connect to the specified server.

SuperWeb [12] was introduced as a global computing platform to utilise the computing resources made available

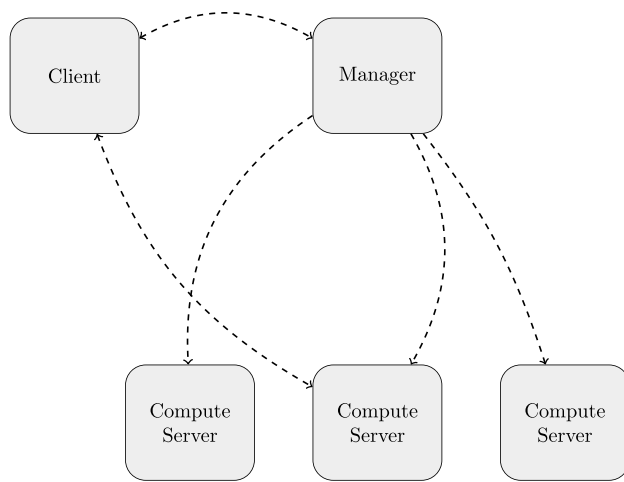


Fig. 1 Atlas architecture

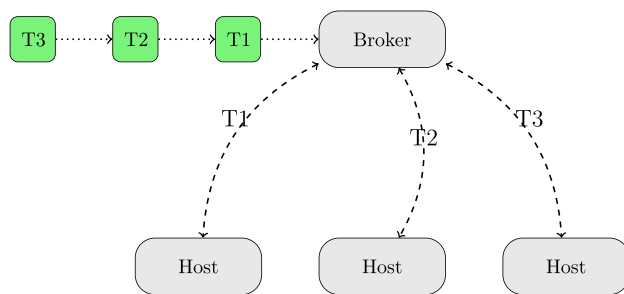


Fig. 2 Three hosts receive client's tasks from the broker

by volunteers connecting to the World Wide Web. The system added three new types of participants: *Brokers*, *Clients*, and *Hosts*. *Broker* is the software that is installed on *Host's* device and is used to organise the computing resources available on this device. *Client* represents the side that has work to be done in the network. By registering in the broker, the host offers his proportion of CPU computing power and RAM memory capacity as a commodity. For example, a host could offer 30% of his CPU time and be more specific about when that 30% should be used, such as between 1 and 6 p.m.

Broker is considered the heart of the SuperWeb architecture, and it can be used by the clients to request computing resources, and by the hosts to offer resources. Figure 2 depicts the interaction between the clients and the hosts when three tasks are submitted to the broker.

Javalin [13] is a global distributed computing environment that was introduced to leverage Java to integrate devices connected over the internet to do common tasks. There are three types of participants, similar to SuperWeb: *Host*, *Client*, and *Broker*. *Client* is a process that seeks computing resources, whereas *Host* is a process that supplies those resources. *Broker*, on the other hand, is an HTTP

server that matches a client's job with a host and coordinates the supply and demand for computing resources. If a host wants to participate, he must launch his Java-based browser and navigate to the task's specific URL.

The clients register their tasks with the broker, which requires generating the equivalent applet; without the need for a server, only a Java-enabled browser can be used to upload the applet to the broker.

The steps to run the applet are as follows:

1. The client uploads the applet to the HTTP server.
2. The client registers the corresponding URL with the broker.
3. The host requests the work from the broker and obtains the URL.
4. The host obtains the applet from the HTTP server and starts running it.
5. After getting results, the host stores them on the HTTP server.
6. After being saved on the HTTP server, the results will be returned to the client.

The term Volunteer Computing was first introduced in 1998 by Luis F. G. Sarmenta in the Bayanihan project [14]. Bayanihan is a web-based framework that enables people to contribute their computers' processing power to solve large-scale parallel problems using Java-enabled browsers by visiting the project's website. Bayanihan's network can accommodate hundreds of thousands or millions of users from all over the world. So, this large number of actively involved volunteers should provide the community with a supercomputing platform far superior to any other supercomputing infrastructure.

Volunteers should be autonomous and anonymous, allowing them to join and leave the network at any time and preventing the administration from accessing their devices. This concept was used in the previous computing system and, more specifically, in Distributed.net. In fact, this concept was altered in Bayanihan, where the new concept of Forced Volunteer Computing was defined. Participants in the Forced Volunteer Computing are neither autonomous nor anonymous. This concept of voluntary computing can be applied to projects that require very accurate outputs, such as making all of a company's computers run the voluntary work in the background when they are idle.

Paid or commercial VC is a new concept introduced in Bayanihan that contradicts the previous VC systems' concept of altruism. This means that, in addition to fully altruistic volunteer computing systems, we can create a commercial volunteer computing network in which we give individual compensation to volunteers who participate in order to attract more and more volunteers to the network, such as by giving random prizes in the form of a lottery or

an amount of money when the work is completed (such as running distributed.net challenges).

Popcorn [15] is also a java-based distributed computing environment infrastructure. It provides a single virtual machine made up of thousands of cores connected via the internet. To encourage participation, this project introduces the concept of a *Market* where trading CPU time occurs. *Selling* and *Buying* of computing power of participants occurs in this market. There are three distinct entities in the Popcorn project:

1. **CPU-time (Buyer):** It represents the API through which the user searches for computing power to purchase.
2. **CPU-time (Seller):** It allows one's CPU time to be used for the parallel program of another. This can be accomplished by using a java-enabled browser to access the official website.
3. **Market:** It represents the online place where the buyers and sellers meet to trade the CPU-Time

There had been many projects that relied on the Popcorn framework, such as Brute-Force Search for code-breaking, simulated annealing, genetic algorithms, planning, and game-tree evaluation.

Charlotte [16] is yet another Java-based parallel programming infrastructure that allows users to create high-performance environments comprised of thousands of connected devices. It creates a virtual machine that isolates the program from the execution environment. This project was established to address issues related to local area parallel computing systems, such as heterogeneity, security, and administrative boundaries.

2.2 Berkeley open infrastructure for volunteer computing

In February 2002, a team led by David Anderson at the University of California, Berkeley's Space Sciences Laboratory (SSL) launched the Berkeley Open Infrastructure For Volunteer Computing (BOINC) [3, 17] project as an open-source middleware system designed to provide a VC environment for performing large distributed applications. The first version of BOINC was released in April 2002, and the first BOINC-based project predictor@home [18] was introduced in June 2002. The first goal of introducing BOINC was to aid the SETI@home [19] project, which analyses radio signals to look for any signs of extraterrestrial life. The framework was then generalised to serve as a platform for other distributed applications in several fields ranging from mathematics to linguistics, medicine, molecular biology, climate science, environmental science, and astrophysics.

Several projects have been launched by employing the BOINC framework. Any volunteer can participate in these

projects; however, he must first download the client application on his device (PC, laptop, or smart phone) and then select the projects that appear to be of interest to him. The computing power of volunteers' devices will be used for this purpose, including CPU and GPU if available, as BOINC supports CUDA and OpenCL [3].

BOINC's internal components are shown in Fig. 3. Multiple components interact with one another via an HTTP-based Remote Procedure Call (RPC) interface. Some components are critical to the BOINC architecture, while others were developed by third parties.

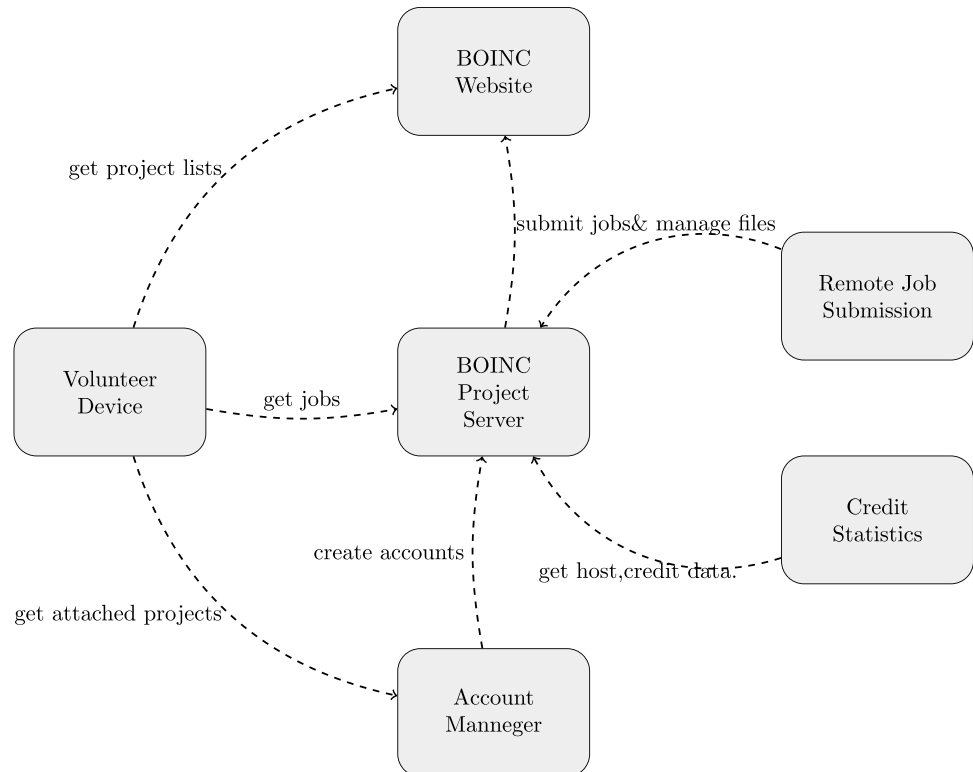
Each BOINC project has its own server on the internet, and anyone interested in participating should go to the project's official website to get a joining link. There could be not only one project to join but a set of research projects. When a volunteer joins a project, the volunteer's device calls RPCs on a regular basis to request a new job, which is sent with all of the necessary files, and when the volunteer completes the allotted job, a new request is issued for a new job, and so on.

When files are submitted between the server and the client in BOINC, encryption and hash function techniques are used to prevent DOS attacks on the server and to ensure the integrity of the submitted files.

Because the volunteer can join any project, there could be multiple projects in which a single volunteer is participating via multiple devices or even one device. Nevertheless, the method of joining and leaving these projects may confuse the user, causing him to lose interest in participating in other projects. BOINC has introduced Account Managers (AM) to address this issue. The user who wants to join a project must first send an RPC request to AM, which will respond with a list of the projects that the user can join. If the volunteer has an account in AM, he can specify his computing preferences, which determine how his computing resources are used. For example, the volunteer can enable CPU throttling [20] which allows the frequency of a CPU to be automatically adjusted in such a way that it conserves power and reduces the amount of heat generated by the CPU. Other preferences, such as limiting network bandwidth and memory usage, can be selected to limit the full control of the volunteer devices.

BOINC has also introduced a mechanism for job replication among volunteers, in which the same job is distributed to two unrelated volunteers' computers, and if the results are identical, the newly joined volunteer is considered trusted, and the result is taken by the server. Otherwise, a third instance is created on another volunteer's computer, and the job is run on it.

BOINC was designed to support device heterogeneity by supporting different processor architectures such as Intel, AMD, Alpha, MIPS, and ARM, which can run 32- or 64-bit applications. Furthermore, the BOINC application works with any operating system, including Linux, Android, MAC OS, and Windows.

Fig. 3 BONIC architecture

2.3 Individual efforts

Individual efforts have been made to deliver new VC frameworks out of the box that simplify use while addressing the limitations of previous frameworks. James Baldassari et al. [21, 22] proposed a new volunteer platform called Simple Light-weight Infrastructure for Network Computing (SLINC) to simplify the process of creating volunteer computing projects. It is a volunteer client–server computing framework designed to be user-friendly and adaptable enough to run any research problem written in any programming language. It also works with a wide range of machine architectures and operating systems.

Erick Lavoie introduced Pando [23] to leverage the participation of the project owner’s social network of friends and family, as the first volunteer computing tool that focuses on personal networks to build a computing environment for personal projects. Pando can be used in both local and global networks of devices, which supports a wide range of applications that represent some of the needs of scientists and programmers in general. Despite being a client–server network, the user’s personal computer can do the server tasks in this project.

Nils Kopal [25] developed in his thesis a new VC framework to reduce the impact of cheaters on volunteer computing. Because the used overlay is a decentralised P2P volunteer network, the suggested chat detection algorithm is distributed across all network peers. Despite the fact that

each peer has a low probability of detecting a cheated result, the network as a whole has a high detection rate.

3 System architecture

There are two versions of our framework. The first is for end users who have no other sub-peers connected, and the second is for users that act as super-peers and have sub-peers connected. While each peer may have at least one version installed, only one version can be active at any given time.

We use TCP-based sockets at the end of each version to ensure secure data exchange between two versions running on different machines. Depending on which version is running, we have the following connection types:

1. **Server-to-Server:** In this type of connection, we get two servers exchanging data over the connection channel. Each server is super-peer to the other peers, however, one of the servers is a sub-peer to the other.
2. **Server-to-Client or Client-to-Server:** In this type of connection, the super-peer either sends the desired data and commands from his side to the sub-peer acting as a client, or the sub-peer sends his data to the super-peer acting as a server.

The block diagram of the two versions is shown in Fig. 4. The internal architecture is made up of several components, each

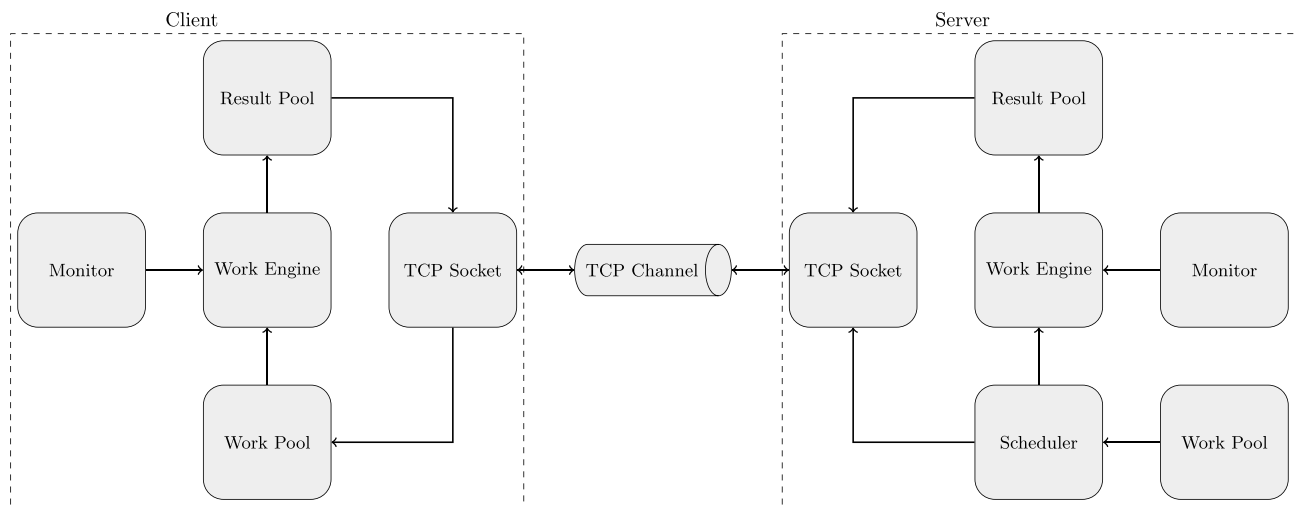


Fig. 4 System architecture

of which performs a specific function, some of which can be performed simultaneously.

The sections that follow go over the functionality of each of our middleware's internal components in great detail.

3.1 Connection sockets

Sockets are used to establish connections between machines connected to the same network. Every socket in the network has its own unique address, which can be determined by three identifiers: **IP address**, **transport protocol**, and **port number**.

Sockets in our network can be assigned the following tasks depending on the type of connected machine:

1. Supper-peer's Socket:

- Send query requests to the connected sub-peer(s).
- Receive responses from the connected sub-peer(s).
- Send jobs to the connected sub-peer(s) to be performed.
- Receive sub-peer(s)'s result packets.

2. Sub-peer's Socket:

- Receive query requests from the connected super-peer(s).
- Send a response to the connected super-peer(s).
- Obtain the assigned job from the connected super-peer.
- Send result packet to the connected super-peer.

3.2 Work engine

It represents a sandbox environment in which the machine performs the assigned work, in the background,

concurrently with other tasks that the operating system is running. To protect the operating system from malicious software, this environment should be completely isolated from the operating system and other users files.

In simulation, we do not have to consider security issues because we represent the work engine as an individual thread that has the task of performing the assigned job given as the number of FLOPS, whereas in real-world implementation, virtual machines are the best implementation of these work engines. In our implementation, however, we can use a Python built-in function called *exec()*, which allows us to run a single instruction or an entire string of Python code.

According to the performance data provided by the monitor thread, the work engine's status can be one of the following:

- **Running:** Work Engine is executing the assigned work.
- **Pausing:** Work Engine is paused due to some external reasons, which are as follows:
 - Work Engine is waiting for additional parameters.
 - Work Engine is temporarily paused when the available computing power falls below a predefined threshold. This status will result in one of two other engine cases: *Running* or *Terminated*.
- **Terminated:** Work Engine will be set to this state if and only if:
 - Work Engine has completed the assigned work and is ready to provide the results to the communication thread that will encapsulate them in a TCP packet and send it to the super-peer.

- The processor's computing availability has been below the predefined threshold for a pre-defined period of time (ten minutes), prompting super-peers to start the task migration process.
- **Hibernated:** Work Engine is awaiting a new job from the super-peer.

Figure 5 depicts the aforementioned status of the work engine when the middleware is active. The beginning status of the engine is always hibernated.

3.3 Global scheduling

The global scheduler is situated at each peer functioning as a server. Its main objective is to determine the workload allocation of the primary task to be distributed among each network sub-peer, ensuring that all devices contribute equally in terms of two key metrics: energy consumption and execution time. This global scheduler, detailed in [24], operates by calculating the maximum computational capacity available to each peer capable of serving as a server. This global scheduler is briefly explained below:

To estimate how many FLOPS each peer's processor i can perform per second, we use Eq. (1):

$$FLOPS_i^{Local} = FLOPS_i^{Peak} \times a_i \times c_i. \quad (1)$$

Where a is the processor availability given as a float number between 0 and 1, $FLOPS^{Peak}$ is the peak processor speed per core, and c is the number of cores.

The accumulated available FLOPS of a given super-peer i with n sub-peers can be calculated as follows:

$$FLOPS_i^{Total} = FLOPS_i^{Local} + \sum_{j=1}^n FLOPS_j^{Local} \quad (2)$$

Now, for a super-peer i with a task t that has an estimated number of FLOPS est_flop_count we can calculate the workload FLOPS that can be allocated to each peer (super-peer and sub-peers) by using Eq. (3):

$$FLOPS_i^{Load'} = est_flop_count_{i,t} / FLOPS_i^{Total} \quad (3)$$

Equation (3) is only applied if all peers have one core per processor with availability 100%, but since each peer i provides different performance depending on the availability a and the number of cores c , we will use Eq. (4) to calculate the workload FLOPS that can be allocated to a peer i :

$$FLOPS_i^{Load} = FLOPS_i^{Load'} \times FLOPS_i^{Peak} \times a_i \times c_i \quad (4)$$

Equation (4) can be easily written in the following form.:

$$FLOPS_i^{Load} = FLOPS_i^{Load'} \times FLOPS_i^{Local} \quad (5)$$

3.4 Work pool

Work pool is available in both middleware versions, but its mission varies depending on the data stored. In the client version, the assigned work received by the socket from the super-peer is stored in Work Pool before being sent to Work Engine to be executed. In the server version, on the other hand, it is used to store the assigned work that will be provided to the scheduler for distribution to all other sub-peers as well as the local super-peer.

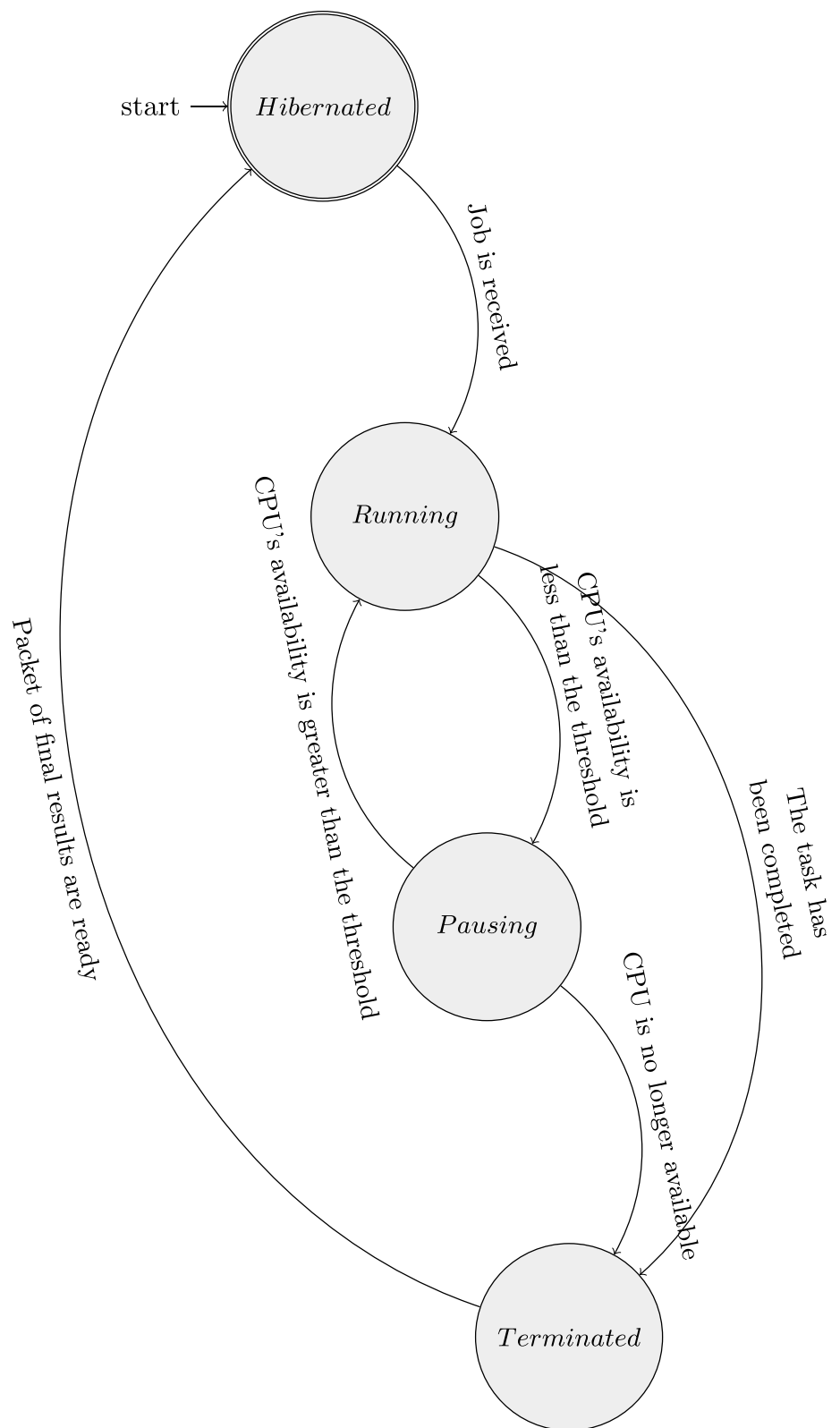
Because each sub-peer receives only one job at a time, Work Pool will only be used to store one job and will not be replaced with a new one unless the current one is completed.

3.5 Results pool

It is the place where all results from both versions of the middleware are saved before being sent to the super-peer. The results are encapsulated in one package known as a Result Packet, along with other information that helps super-peers identify the contributed peers and their job completion status. The following is the structure of Result Packet:

- **Identifier of the sub-peer:** It is an identifier of the peer who is currently performing the given tasks. This identifier is used to distinguish between the network's participating peers. During the simulation stage, we assign each peer a unique string to ensure that the super-peer can determine who performed the assigned task. In real implementation, we can use a combination of IP addresses and MAC addresses to identify each peer.
- **Identifier of the super-peer:** It is an identifier of the peer to which the result packet should be routed. Similarly, this identifier should be unique to this peer so that the sub-peer knows exactly the final destination of its result packet.
- **Participants:** All super-peers keep track of their sub-peers that participated in performing the assigned jobs. This field's value is only the identifier of the current peer if it is a final sub-peer.
- **Execution Time:** We store in this field the duration required for the peer to finish the given task. All super-peer will collect all peers' execution times to ensure that the global scheduler's objectives are met.
- **Completion Status:** It is used to track whether or not the job was completed successfully. This status allows the super-peer to determine whether or not the corresponding sub-peer is actively participating in performing the

Fig. 5 The four status of work engine



assigned job, allowing it to determine which sub-peers are ready for the task migration procedure.

- **Leftover Work:** Peers save a portion of the work that could not be completed due to a lack of computing power.

3.6 Monitoring routine

It is an independent thread that continuously tracks CPU and RAM usage every 60 s in the background and notifies the work engine if the usage rate exceeds a predefined threshold for ten consecutive minutes, at which point the device declares itself busy and therefore unavailable to complete the assigned task. To avoid using the CPU at its highest power and to limit its overheating to a high level, we also put a pre-defined limit on the accepted availability at (50%) below which the CPU will suspend the processing of the assigned task if its availability drops below it.

4 Network architecture

Our network utilizes a structured overlay framework, wherein although the middleware is distributed randomly, the resultant overlay adopts a tree-like topology. This approach minimizes the routing effort for each peer, as they can efficiently locate other peers or resources within the network without resorting to indiscriminate message flooding. Achieving this in a structured P2P overlay involves storing pertinent peer-related data in accessible lookup tables shared among all network participants. These tables are flexible, allowing for immediate modifications to registered data.

We provided the simulator with trace files, ensuring network heterogeneity by allocating them to each device. These files encapsulate performance metrics delineating the distinct behavior of each host. Our simulation stage begins by selecting an appropriate dataset of trace files. GWA-T-13 Materna [1] stands out as an ideal option for this purpose. It encompasses performance metrics gleaned from three databases spanning numerous devices dispersed across Materna Data Centres in Dortmund, Germany, over a three-month span. Every trace file is structured in a columnar format, with each column denoting a distinct performance metric. Each row within these files encapsulates various metrics recorded at a specific time instance.

- **Timestamp:** Milliseconds since January 1, 1970.
- **CPU cores:** The machine's virtual CPU core count.
- **CPU capacity:** The number of cores (FLOPS) times the speed per core gives the CPU's capacity in MHZ.
- **CPU usage as the speed:** This is given in millihertz (MHZ).

- **CPU usage as percentage:** This is expressed as a percentage.
- **Memory provisioned:** Memory capacity of the virtual machine in KB.
- **Size of memory usage:** The portion of memory that is used in KB.
- **percentage of memory usage:** The portion of memory that is used in %.
- **Disk size:** Size of virtual hard disc in GB.
- **Disk write throughput:** Writing bandwidth is specified in KB/Second.
- **Data transfer rate over the network for received data:** Network downloading speed in KB/Second.
- **Data transfer rate over the network for transmitted data:** Network uploading speed in KB/Second.

We only rely on CPU usage, which is expressed as a percentage, to determine whether or not the CPU is available. However, we calculate the availability percentage rather than the CPU usage percentage.

Figure 6 depicts the available CPU power in percent as recorded on November 5, 2015, for 24 h. We can see that the CPU was mostly idle throughout the day, with the exception of the time between 10:00:00 and 14:00:00, when the observed availability shows an unstable availability rate, causing it to fall, reaching its lowest value at 11:10:00. In our simulation, we fixed a 50% availability threshold, below which the CPU must stop running the assigned work. We use this strategy to avoid intrusive use of the host's CPU and to reduce CPU heat.

Each peer has its own behaviour, which is determined by the availability of each processor and the number of cores. This reflects on the power consumption that will be consumed in each node, but this does not imply that the consumed power increases if the processor uses an extra number of cores; for example, a processor with one core may consume more energy than a processor with two cores if the availability of the first processor is greater than the availability of the second.

Figure 7 depict our network used in simulation. The network was built entirely randomly to reflect the viral spread of our middleware among peers. For both networks, there is one peer from which the middleware is shared with other sub-peers, and among these sub-peers, there is at least one peer that shares the middleware (project invitation) with other sub-peers, and so on for the rest of the network.

Table 1 categorises the peers in both networks based on the number of cores they have. Also, the different power parameters for each peer are given to each group.

The precise maximum computing power, quantified in terms of FLOPS, available within each network cannot be definitively assumed. This variability is directly linked to

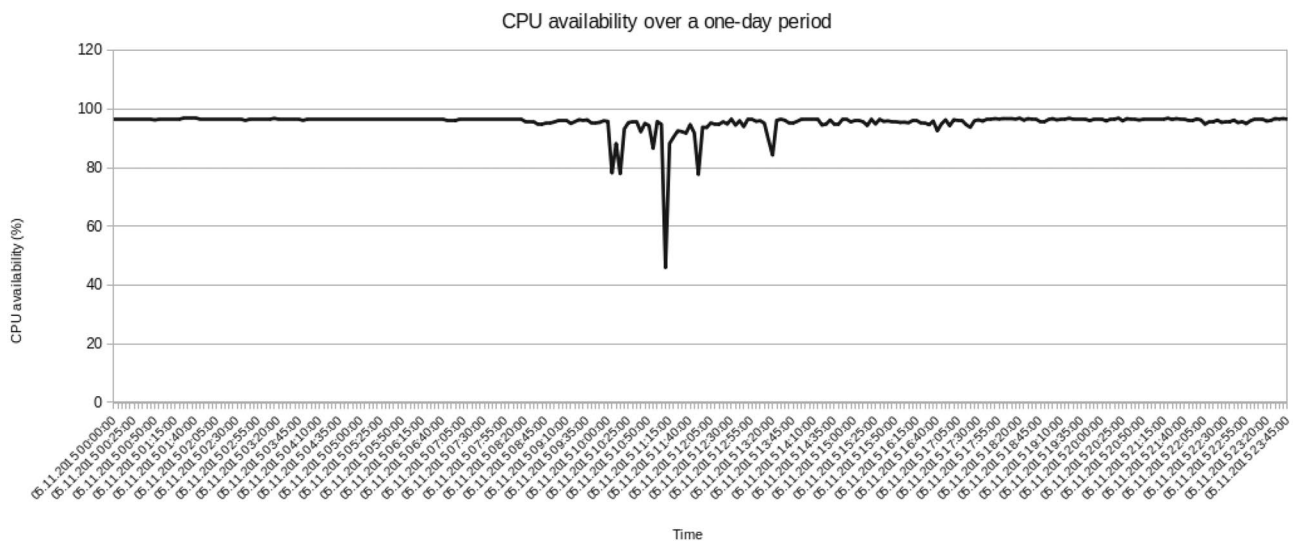


Fig. 6 Example of CPU availability on a single day

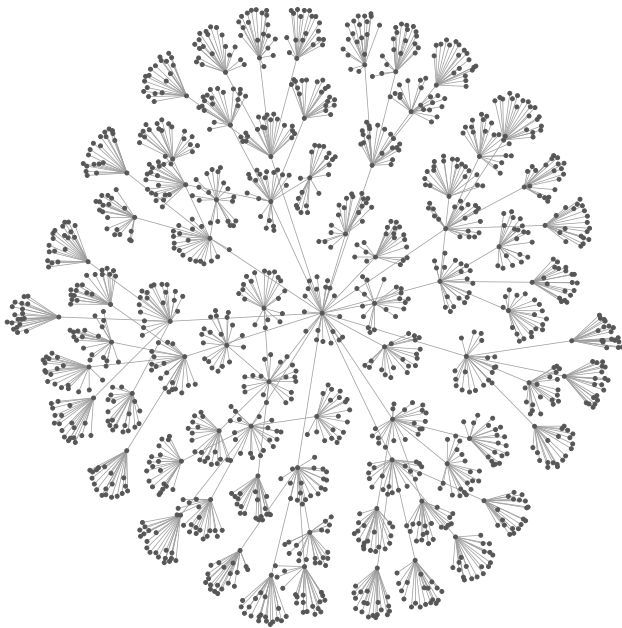


Fig. 7 The network overlay of 1500 connected peers

the availability profile associated with each peer across these two networks. Nonetheless, if we hypothetically presume that each peer remains consistently available at 100%, we can straightforwardly compute the FLOPS. This calculation yields a total of 19.11 TeraFLOPS, derived from a cumulative core count of 3822 cores.

5 Simulation results

In addition to monitoring the execution time, we tracked the total energy consumption across the entire network for each task size to observe consumption trends with increasing task sizes. However, as power consumption encompasses two components - static, occurring during peer idleness, shutdown, or operation, and dynamic, arising when the CPU is in use - we meticulously monitored the network's overall power usage in both of these categories.

Table 2 demonstrates the maximum execution time and consuming power for each of the tested task sizes. We can see the consumed power rapidly increasing as the task size

Table 1 Peer specs according to the number of cores in the processor

cores count/peer	Peers number	Power Consumption Parameters (Watt)				
		Peak GFLOPs/ core	Idle	Allcores	Epsilon	Off core
One-core CPU	202	5	100	140	120	10
Two-core CPU	953	5	100	160	120	10
Four-core CPU	242	5	100	200	120	10
Six-core CPU	39	5	100	240	120	10
Eight-core CPU	64	5	100	280	120	10

Table 2 The maximum execution time and power consumption depend on the size of the task

Task size (FLOPS)	Maximum execution time (Seconds)	Power consumption(Joules)		
		Used peers	Idle peers	Total
10^{12}	1	274684.71	1596.73	276281.43
10^{13}	1	274684.71	1596.73	276281.43
10^{14}	5.48	1204530.1	6527.31	1211057.37
10^{15}	54.82	11440153.2	60802.54	11500955.74
10^{16}	574.72	121173136.14	657810.45	121830946.59
10^{17}	6063.79	1528145057.41	8102212.63	1536247270.04

increases. It should be noted that the execution time shown in Table 2 is the maximum accumulated time that a peer takes to complete the assigned task and does not account for delays that occur when the peer becomes unavailable or is waiting for data from other peers.

Understanding the ratio of power consumption between idle and active states is crucial. Figure 8 illustrates the percentage of total power consumption attributed to active and idle states in each scenario. Remarkably, even with increasing task sizes, the ratio of power consumed between idle and active states has remained relatively constant.

Approximately 99.95% of the total power was consumed during active execution of assigned tasks, while roughly 0.5% was consumed during idle periods.

Figure 9 displays the RSD percentage of the execution time for each of the tested scenarios. We can see that RSD values are negligible in all cases, with the exception of the experiments where 10 petaFLOPS and 100 petaFLOPS were distributed among all peers, where RSD values are too small (2.4 percent and 5 percent, respectively). This demonstrates that across all scenarios, the execution times of all peers are tightly grouped around their mean.

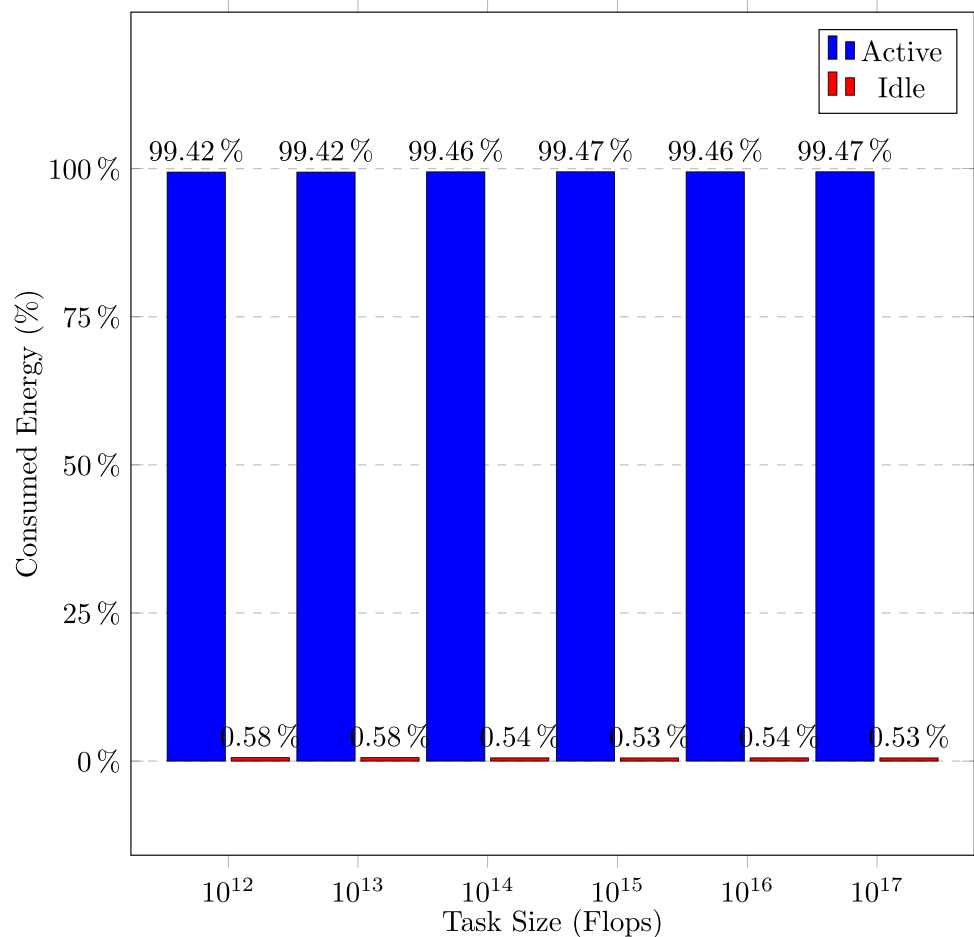
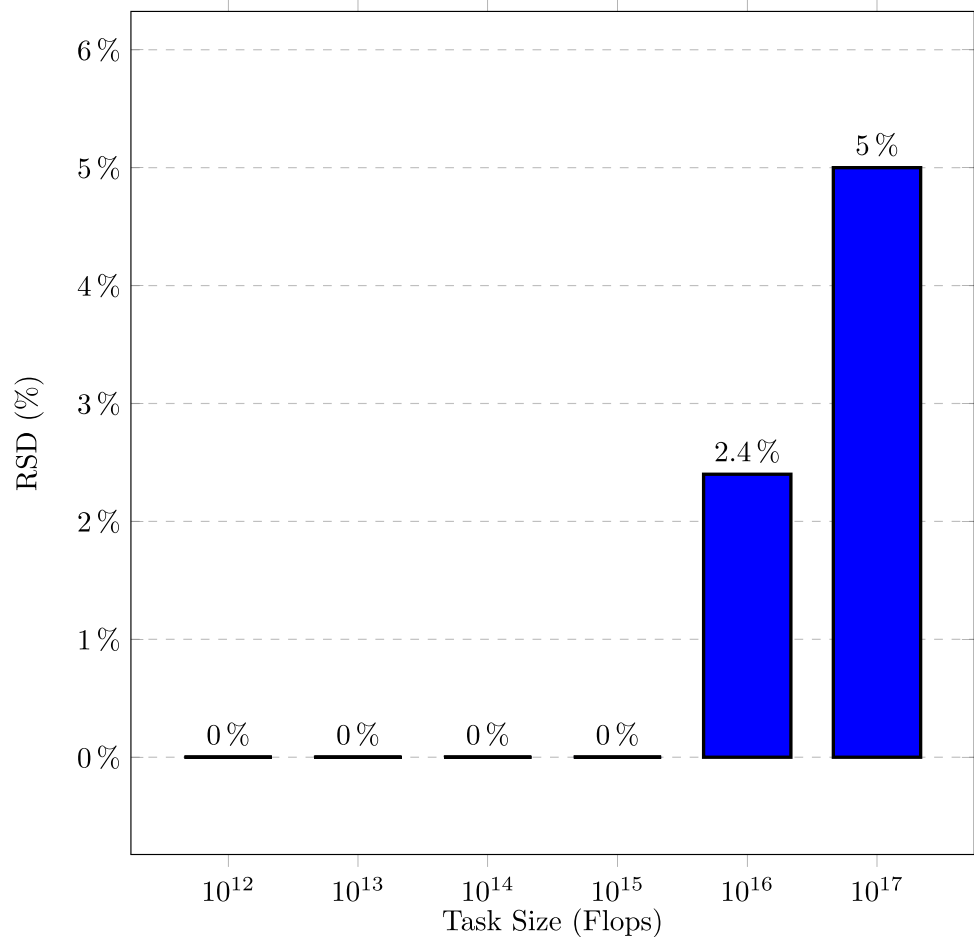
Fig. 8 Total consumed power when peer are idle and active

Fig. 9 RSD of the execution time at each scenario

The RSD percentage of the power consumption for each scenario are shown in Fig. 10. When we distribute 100 petaFLOPS, the increase in RSD values reaches about 1.2 percent, but RSD values still show low percentages, demonstrating that peers utilise power equally.

We calculated the speedup in each of each task size provided to the network to compare how much improvement we get in the execution time when we run a task of a specific size in P processors.

The term "speedup" is somewhat ambiguous in this context. The normal calculation of the speedup is performed when all processors are homogeneous. In this case, the speedup is simply calculated as $T1/Tp$, where $T1$ is the time it takes for a program to run on one processor and Tp is the time it takes for the same program to run on p processors.

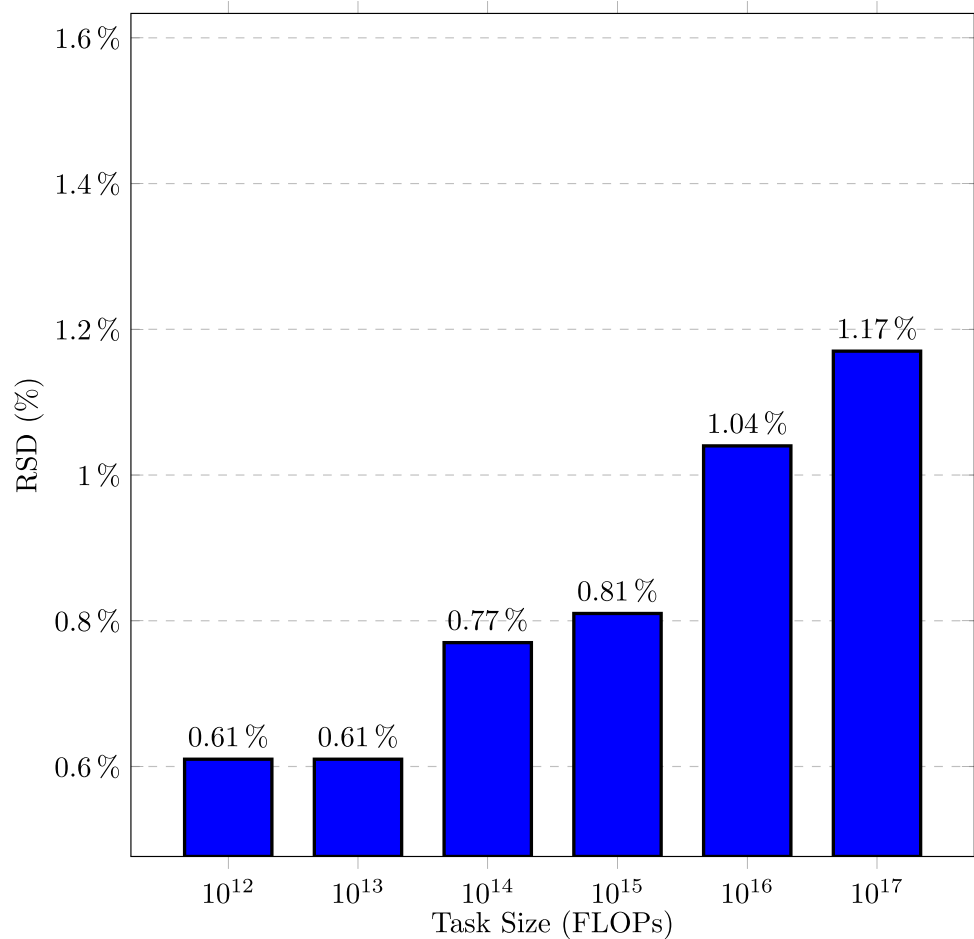
However, in a network of heterogeneous processors, each of which provides its own computing power based on the number of cores and availability, we cannot predict what the correct value of $T1$ would be.

In fact, we can give an estimate of $T1$ when we retrieve the time it takes each processor p_k to calculate the assigned task as follows:

$$T1(p_1, \dots, p_k) = \frac{p_1 T1(1) + \dots + p_k T1(k)}{p_1 + \dots + p_k} \quad (6)$$

The calculated $T1$ is the theoretical average execution time. Practically, however, this appears to be an impractical solution for tasks with large sizes because we must implement the same task on each processor to perform Eq. (6). Instead, for our scenarios, we have chosen one processor with two cores and 70% availability to estimate $T1$. This processor will be able to perform 7 GFLOPS at full speed with theoretical Peak GFLOPS/core = 5.

Figure 11 depicts the speedup we calculated in all of task sizes using the above concept of $T1$ and the execution time values shown in Table 2. We can see that when we run the first three scenarios, the speedup increased linearly, indicating that our system improves the speedup when performing tasks with sizes less than one petaFlops. However, when we run other tasks that were larger than one petaFlops, the speedup began to decrease, indicating that the system's current computing capability could not improve the speedup when we increased the task size to be larger than the accumulated computing power that it could

Fig. 10 RSD of the power consumption at each scenario

provide. Nonetheless, the speedup values are far too high in all scenarios.

Similarly, we calculated Efficiency, which is the ratio of the speedup to the number of computing units. It is used to calculate how much computing power is actually used in computing. As a general rule, the efficiency should be greater than 0.5. Figure 12 shows the efficiency we calculated in all scenarios. We can notice that this curve looks identical to what we got in Fig. 11. This is because the number of cores is fixed, and only the task size changes.

We can see that our system improves its efficiency when performing tasks with sizes less than one petaFlops. When we run other tasks larger than one petaFlops, however, the efficiency began to decline, implying that the system's current computing capability could not improve efficiency when the task size was increased to be larger than the accumulated computing power that it could provide.

The only way to ensure that efficiency and speedup are constantly increasing as the size of the task increases is to increase the number of computing units in the system. Also, we must ensure that the availability of most of these computer units is sufficient so that we can use as much computing power as possible to perform our tasks.

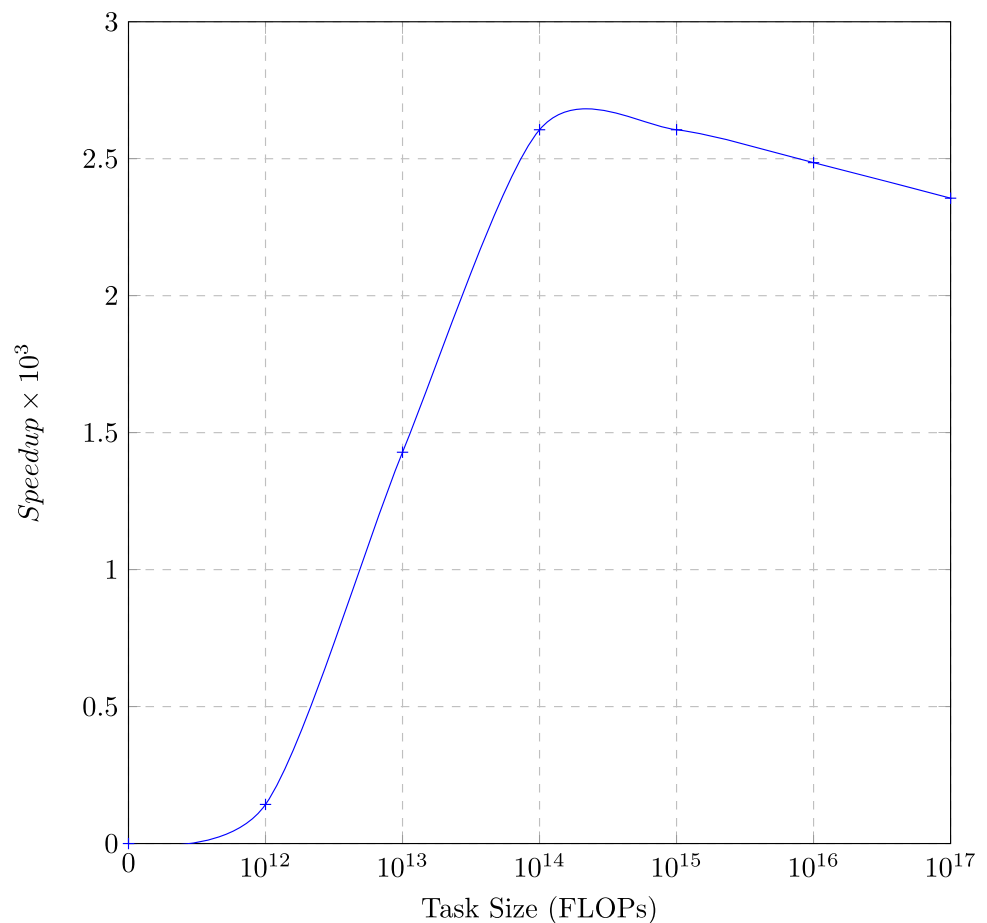
6 Conclusions

In conclusion, this paper has introduced a novel Volunteer Computing (VC) framework designed to overcome the limitations of traditional Client–Server architectures prevalent in existing VC systems. Our framework employs a P2P architecture where each volunteer node acts simultaneously as a client and server. Volunteers acting as servers can execute a global scheduler that facilitates equitable task allocation among all participants, resulting in minimal Relative Standard Deviation (RSD) values for errors. Through extensive modeling and simulation, our framework has been rigorously tested. The simulation results demonstrated its feasibility and effectiveness in various scenarios, underscoring its potential for real-world applications.

7 Future scope

The proposed framework can be extended to provide more efficient performance, and our approach can always be improved to a more advanced version. Some ideas for the future have been proposed as a result of recent approaches:

Fig. 11 The theoretical speedup at each task size



7.1 Availability prediction

Churn effect or churn rate affects large-scale volunteer systems due to resource failure and people joining and leaving the network. A higher churn rate indicates that we have fewer resources that can remain in the network for an extended period of time, whereas a lower churn rate indicates that we have a network with a large number of available resources.

Although task migration is thought to be the best method for reducing churn, it has some disadvantages that must be taken into consideration. These disadvantages related to connection costs, power consumption, and task execution times.

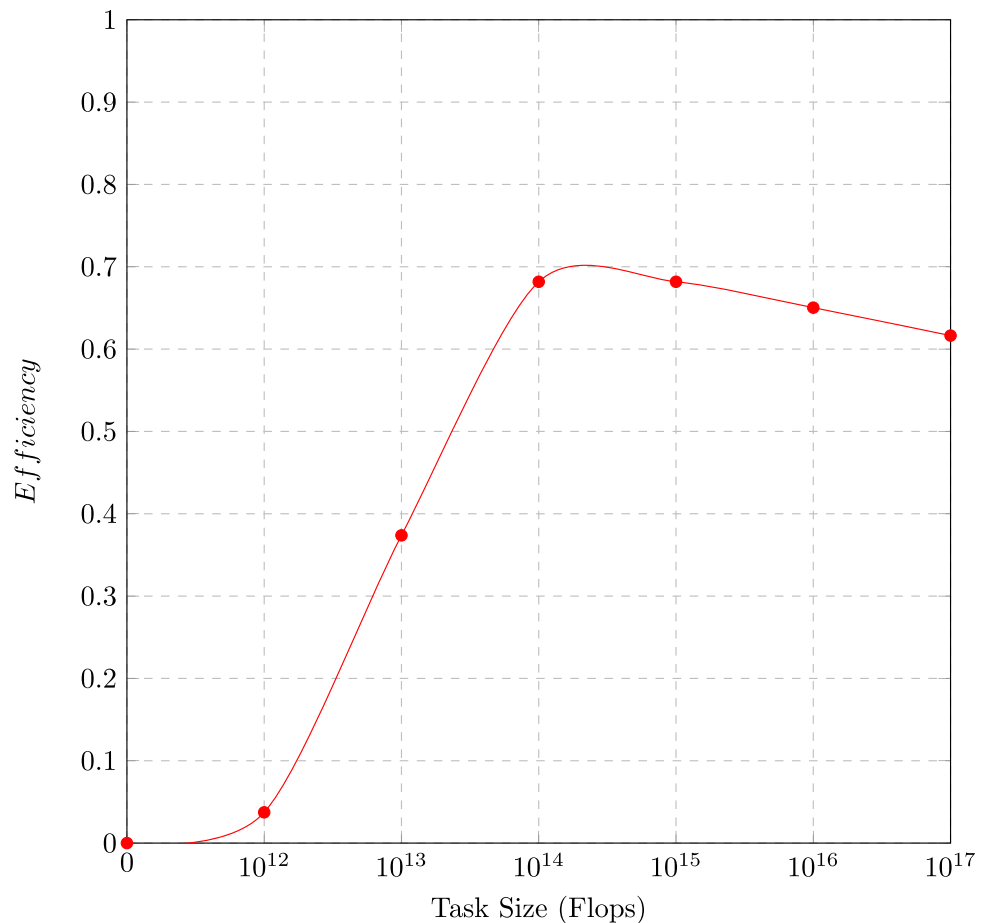
- In terms of cost, the server must now search for unused extra resources to execute the work left by the departing ones and compensate for their absence. There will also be a communication cost, which may or may not be significant.
- In terms of execution time, the server must reschedule unfinished works to be executed using other available resources. And this will add extra time to the expected execution time.

- In terms of power consumption, adding additional network resources to participate in computing processes, as well as migrating the works left over from the departed computing resources, will increase total power consumption beyond what can be tolerated.

We believe that using prediction models to estimate these peers' availability in advance will reduce the number of migration processes. So, we suggest using a machine-learning-based model for predicting the long-term availability of computing resources in large-scale, distributed systems.

7.2 Support different types of processing units

We used a dataset of tracing records from Materna Data Centres in Dortmund, Germany, to test the proposed algorithms in this paper. The real implementation, on the other hand, runs on various devices running either Windows or Ubuntu. Despite the device's heterogeneity, the following points should be addressed in a new version of our middleware:

Fig. 12 The theoretical efficiency at each task size

- The middleware should run in addition to traditional computing units such as GPUs and FPGAs, as well as other configurable processing units that could be plugged in or even integrated within the same processor. As an example, the laptop Novena is powered by a Freescale i.MX6Q ARM processor and a Xilinx Spartan 6 FPGA [26].
- Other devices, such as smart phones and smart watches, are still not supported in the current version of middleware. This device is equipped with various processor architectures and operating systems. However, because these devices use integrated power sources, power consumption will be the most important consideration when developing middleware for these devices.

These two points appear to be more technical, and implementing a middleware framework to support any device architecture or operating system will not require a lot of effort or time.

Data availability Not applicable.

Declarations

Conflict of interest The authors declare that there are no Conflict of interest regarding the publication of this paper.

References

1. “Gwa-t-13 materna.” <http://gwa.ewi.tudelft.nl/datasets/gwa-t-13-materna>. Accessed: 2022-01-31
2. Padua D (2011) Network of workstations. Springer, Boston, pp 1303–1303
3. Anderson DP (2019) Boinc: A platform for volunteer computing. *J Grid Comput* 18:99–122
4. Anderson DP (2021) “Globally scheduling volunteer computing,” *Future Internet*, vol. 13, no. 9,
5. “Great internet mersenne prime search gimps finding world record primes since 1996.” <https://www.mersenne.org/prime/s/?press=M1398269>. Accessed: 2022-09-1
6. “distributed.net the organization.” https://www.distributed.net/Main_Page. Accessed: 2022-01-30
7. Brecht T, Sandhu H, Shan M, Talbot J (1996) Paraweb: towards world-wide supercomputing. In: Proceedings of the 7th Workshop on ACM SIGOPS European Workshop: Systems Support for Worldwide Applications, EW 7, (New York, NY, USA), p. 181–188, Association for Computing Machinery
8. “The java parallel runtime system (jprs).” <https://cs.uwaterloo.ca/~brecht/papers/html/paraweb/node5.html#SECTION00032000000000000000>. Accessed: 2022-09-1
9. “The java parallel class library (jpccl).” <https://cs.uwaterloo.ca/~brecht/papers/html/paraweb/node4.html#SECTION00031000000000000000>. Accessed: 2022-09-1
10. Baldeschwieler JE, Blumofe RD, Brewer EA (1996) Atlas: An infrastructure for global computing. In Proceedings of the 7th

- Workshop on ACM SIGOPS European Workshop: Systems Support for Worldwide Applications, EW 7, (New York, NY, USA), p. 165-172, Association for Computing Machinery
11. Blumofe RD, Joerg CF, Kuszmaul BC, Leiserson CE, Randall KH, Zhou Y (1995) Cilk: An efficient multithreaded runtime system. In: Proceedings of the Fifth ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPOPP '95, (New York, NY, USA), p. 207-216, Association for Computing Machinery
 12. Alexandrov AD, Ibel M, Schauser KE, Scheiman CJ (1997) Superweb: research issues in java-based global computing. *Concurr Pract Exp* 9:535–553
 13. Christiansen BO, Cappello PR, Ionescu MF, Neary MO, Schauser KE, Wu D (1997) Javelin: internet-based parallel computing using java. *Concurr Pract Exp* 9:1139–1160
 14. Sarmenta LF, Hirano S (1999) Bayanihan: building and studying web-based volunteer computing systems using java. *Futur Gener Comput Syst* 15(5):675–686
 15. Nisan N, London S, Regev O, Camiel N (1998) Globally distributed computation over the internet - the popcorn project. In: Proceedings of the The 18th International Conference on Distributed Computing Systems, ICDCS '98, (USA), p. 592, IEEE Computer Society
 16. Baratloo A, Karaul M, Kedem ZM, Wijckoff P (1999) Charlotte: Metacomputing on the web. *Fut Gener Comput Syst* 15:559–570
 17. "Compute for science." <https://boinc.berkeley.edu/>. Accessed: 2022-09-05
 18. Taufer M, An C, Kerstens A, Brooks C (2005) Predictor@ home: a protein structure prediction supercomputer based on public-resource computing. In: 19th IEEE International Parallel and Distributed Processing Symposium, pp. 8
 19. Seti@home. <https://setiathome.berkeley.edu/>, note = Accessed: 2022-09-05
 20. Compute for science. <https://www.ibm.com/docs/en/tivoli-monitoring/6.3.0?topic=configuration-cpu-throttling>. Accessed: 2022-09-05
 21. Baldassari J, Finkel D, Toth D (2006) SlinC: A framework for volunteer computing. 04
 22. Baldassari JD. Master's thesis
 23. Lavoie E, Hendren L (2019) Personal volunteer computing. In: Undefined command of the 16th ACM International Conference on Computing Frontiers, CF '19, (New York, NY, USA), p. 240-246, Association for Computing Machinery
 24. Saleh E, Shastry C (2022) A new approach for global task scheduling in volunteer computing systems. *Int J Inf Technol*. <https://doi.org/10.1007/s41870-022-01090-w>
 25. Kopal N (2018) Secure volunteer computing for distributed cryptanalysis. PhD thesis, University of Kassel, Germany
 26. Want a laptop with a fpga? you're in luck!. <https://community.element14.com/technologies/fpga-group/t/forum/24354/want-a-laptop-with-a-fpga-you-re-in-luck>. Accessed: 2022-10-22
- Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.